EXPERIMENT NO. 4

AIM-To perform INNER JOIN, LEFT JOIN, SELF JOIN, 3-WAY JOIN, Correlated Subquery, EXISTS, simulated INTERSECT and EXCEPT, and compare execution plans using EXPLAIN.

- CREATE TABLES WITH DATABASES

1. DEPARTMENT TABLE WITH VALUES

```sql
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(30)
);

INSERT INTO Department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance'),
(4, 'Marketing'),
(5, 'Operations');
```

2. PROJECT TABLE WITH VALUES

```sql
CREATE TABLE Project (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(50),
    Budget INT,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Project VALUES
(101, 'Cloud Migration', 800000, 1),
(102, 'AI Chatbot', 1200000, 1),
(103, 'Employee Portal', 500000, 2),
(104, 'Payroll System', 700000, 3),
(105, 'Market Analysis', 600000, 4),
(106, 'Digital Campaign', 900000, 4),
(107, 'Supply Chain', 1000000, 5),
(108, 'Inventory Management', 750000, 5);
```

3. EMPLOYEE TABLE WITH VALUES

```sql
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(30),
    Salary INT,
    DeptID INT,
    ProjectID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
    FOREIGN KEY (ProjectID) REFERENCES Project(ProjectID)
);

INSERT INTO Employee VALUES
(1,'Aarav',55000,1,101),
(2,'Priya',65000,1,102),
(3,'Rahul',48000,1,101),
(4,'Ananya',72000,1,102),
(5,'Karan',60000,1,101),
(6,'Neha',58000,1,102),

(7,'Riya',45000,2,103),
(8,'Aditya',52000,2,103),
(9,'Simran',48000,2,103),
(10,'Vikas',62000,2,103),
(11,'Pooja',55000,2,103),
(12,'Manish',50000,2,103),

(13,'Rohan',70000,3,104),
(14,'Isha',68000,3,104),
(15,'Arjun',75000,3,104),
(16,'Meera',62000,3,104),
(17,'Nitin',58000,3,104),
(18,'Kavya',64000,3,104),
```

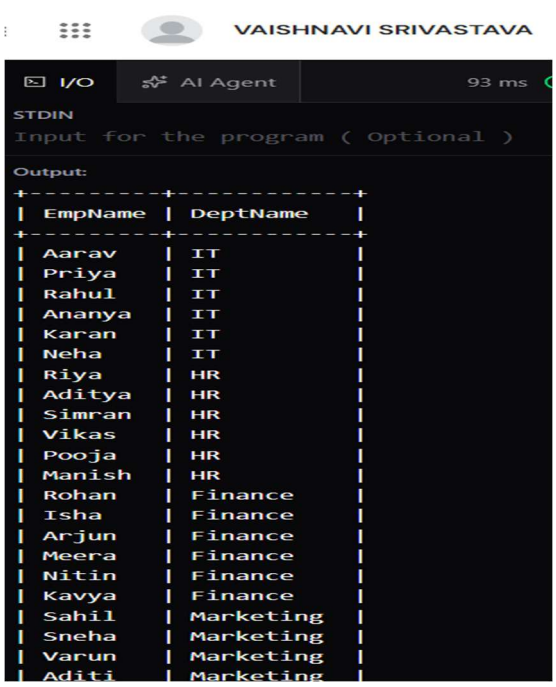
```sql
(19,'Sahil',54000,4,105),
(20,'Sneha',60000,4,106),
(21,'Varun',57000,4,105),
(22,'Aditi',69000,4,106),
(23,'Mohit',62000,4,106),
(24,'Tanya',56000,4,105),

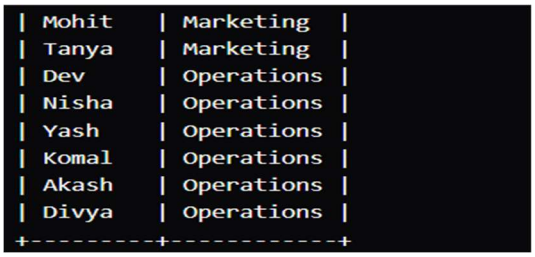
(25,'Dev',59000,5,107),
(26,'Nisha',63000,5,108),
(27,'Yash',55000,5,107),
(28,'Komal',61000,5,108),
(29,'Akash',67000,5,107),
(30,'Divya',58000,5,108);
```

- ***REQUIRED QUERIES-***

1. **INNER JOIN**

```
---- SELECT E.EmpName, D.DeptName
     FROM Employee E
     INNER JOIN Department D
     ON E.DeptID = D.DeptID;
```



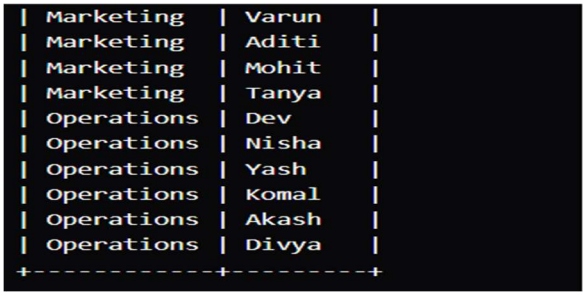


---

2. **LEFT JOIN**

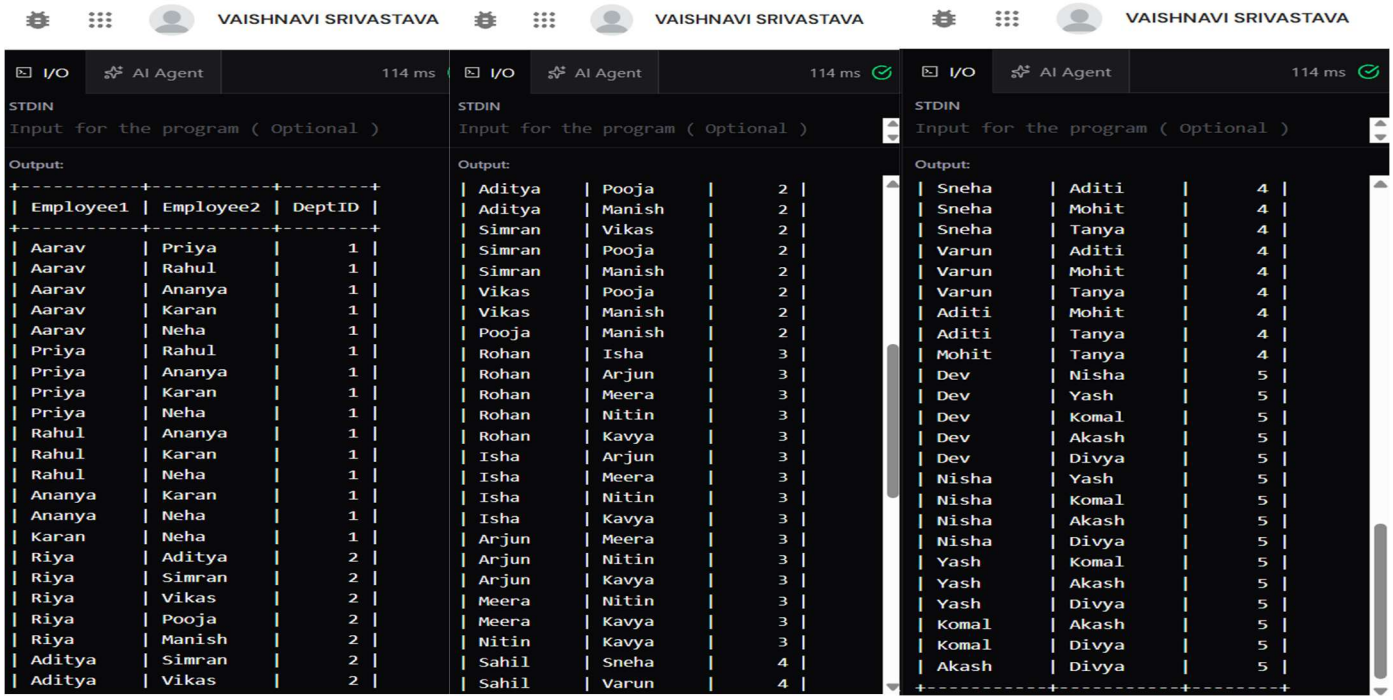
```
---- SELECT D.DeptName, E.EmpName
     FROM Department D
     LEFT JOIN Employee E
     ON D.DeptID = E.DeptID;
```





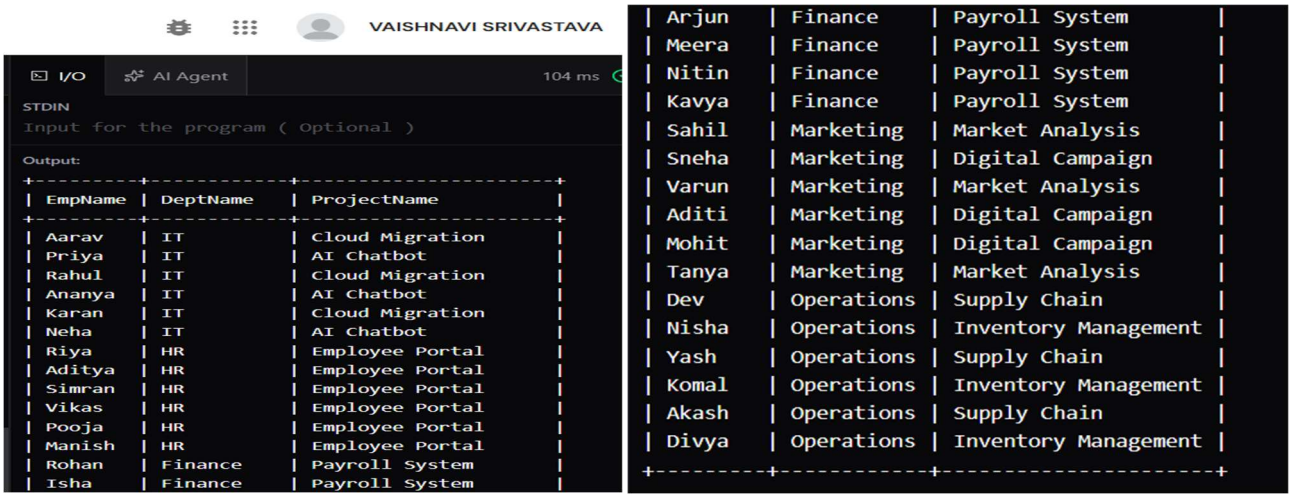
3.**SELF JOIN**

```
---- SELECT E1.EmpName AS Employee1,
     E2.EmpName AS Employee2,
     E1.DeptID
     FROM Employee E1
     JOIN Employee E2
    ON E1.DeptID = E2.DeptID
    AND E1.EmpID < E2.EmpID;
```



4. **3-WAY JOIN**

```
SELECT E.EmpName, D.DeptName, P.ProjectName
FROM Employee E
JOIN Department D
ON E.DeptID = D.DeptID
JOIN Project P
ON E.ProjectID = P.ProjectID;
```



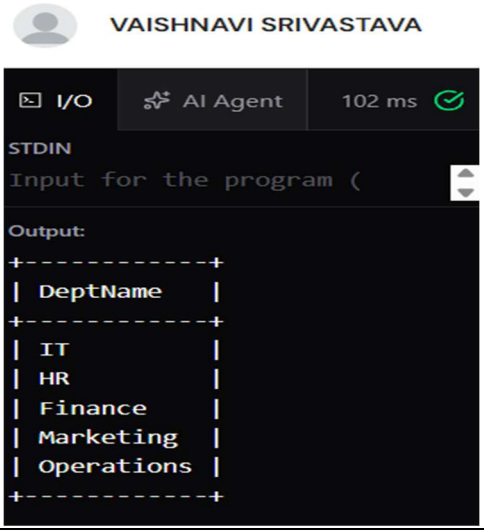
5.CORRELATED JOINS

```
---- SELECT E.EmpName, E.Salary, E.DeptID
      FROM Employee E
      WHERE E.Salary >
     (
     SELECT AVG(E2.Salary)
     FROM Employee E2
     WHERE E2.DeptID = E.DeptID
     );
```



6.**EXIST JOIN**

```
SELECT D.DeptName
FROM Department D
WHERE EXISTS
(
  SELECT 1
  FROM Employee E
  WHERE E.DeptID = D.DeptID);
```



7.SIMULATED INTERSECT

```
SELECT EmpName
FROM Employee E
WHERE E.Salary > 60000
AND EXISTS (
  SELECT 1
  FROM Project P
  WHERE P.ProjectID = E.ProjectID
  AND P.Budget > 700000
);
```

VAISHNAVI SRIVASTAVA



8.SIMULATED EXCEPT

```
SELECT EmpName
FROM Employee
WHERE EmpName NOT IN (
  SELECT E.EmpName
  FROM Employee E
  JOIN Project P
  ON E.ProjectID = P.ProjectID
  WHERE P.ProjectName = 'Cloud Migration'
);
```

VAISHNAVI SRIVASTAVA



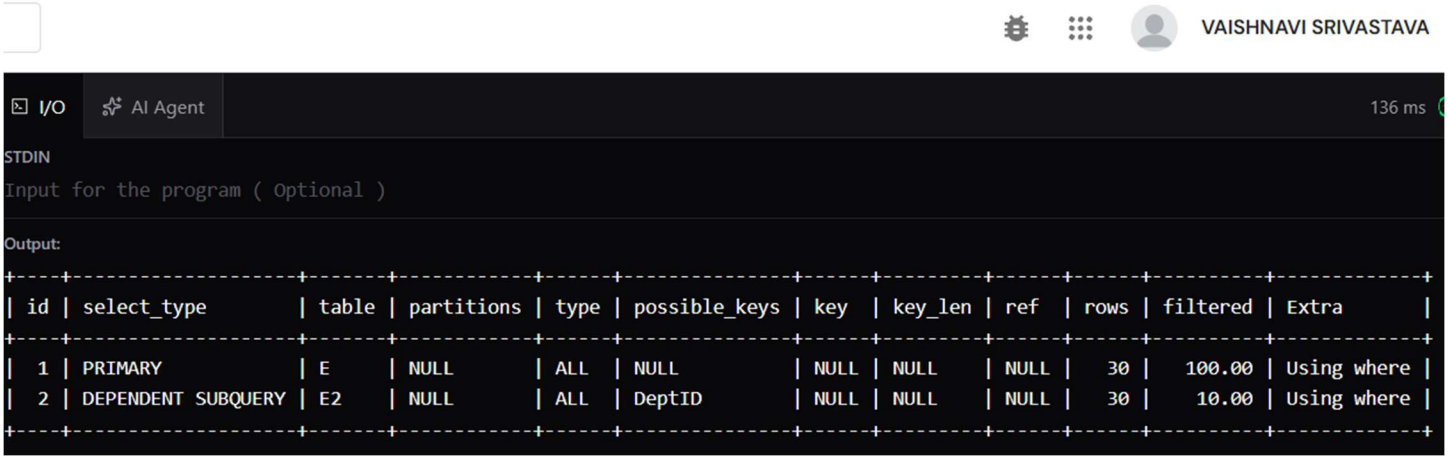
9.**EXPLAIN-INNER JOIN**

```
EXPLAIN
SELECT E.EmpName, D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```



EXPLAIN – Correlated Subquery

```
EXPLAIN
SELECT E.EmpName, E.Salary
FROM Employee E
WHERE E.Salary >
(
  SELECT AVG(E2.Salary)
  FROM Employee E2
  WHERE E2.DeptID = E.DeptID
);
```



COMPARISION

Feature	INNER JOIN	Correlated Subquery
Tables involved	Employee + Department	Employee + Employee
Main operation	JOIN	Subquery
Relationship	Matches rows using DeptID	Inner query depends on outer query
Execution	Database creates a join plan	Inner query is related to each outer row
Complexity	Generally simpler	Can require more repeated work
EXPLAIN	Shows join/access strategy	Shows outer query and dependent subquery
Use	Combining related tables	Comparing each row with its group/related data

Result

The required **INNER JOIN, LEFT JOIN, SELF JOIN, 3-WAY JOIN, correlated subquery, EXISTS, simulated INTERSECT and simulated EXCEPT** queries were successfully implemented. The execution plans were examined using EXPLAIN to understand how the database executes the queries.